

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task 6.2

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Node.js Interactive File System Manager

2. AIM

To implement a Node.js file system manager using the fs module.

3. OBJECTIVE

The objectives of this experiment are:

- Accept user input for filename and content.
 - Create, read, and append to files dynamically.
-

4. THEORY

The Node.js fs module allows interaction with the file system. Readline can be used for interactive terminal input.

5. ALGORITHM / PROCEDURE

1. Require fs and readline modules.
 2. Prompt user for filename and initial content.
 3. Write to the file and read it back.
 4. Prompt for append content, append it, and read the final output.
-

6. PROGRAM

File: Source Code

fileManager.js

```
const fs = require("fs");
const path = require("path");
const readline = require("readline");

// Configure Readline Interface for Interactive Terminal User Input
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(query) {
  return new Promise((resolve) => rl.question(query, resolve));
}

async function runFileManager() {
  console.log("=====");
  console.log("  Node.js Interactive File System Manager");
  console.log("=====\\n");

  try {
    // 1. Accept filename and initial content from user (interactive or CLI args fallback)
    let filename = process.argv[2];
    let initialContent = process.argv[3];
    let appendContent = process.argv[4];

    if (!filename) {
      filename = await askQuestion("Enter filename to create (e.g. sample.txt): ");
    }
    if (!filename.trim()) {
      filename = "user_document.txt";
    }

    const filePath = path.isAbsolute(filename) ? filename : path.join(__dirname, filename);

    if (!initialContent) {
      initialContent = await askQuestion("Enter initial content to write into file: ");
    }

    // 2. Create / Write File Operation
    console.log(`\\n[STEP 1] Creating and writing to '${filename}'...`);
    fs.writeFileSync(filePath, initialContent, "utf8");
    console.log(`[SUCCESS] File '${filename}' written successfully (${Buffer.byteLength(initialContent)} bytes).`);

    // 3. Read Contents Operation
    console.log(`\\n[STEP 2] Reading contents of '${filename}'...`);
    const currentContent = fs.readFileSync(filePath, "utf8");
    console.log("----- Current Content -----");
    console.log(currentContent);
    console.log("-----");

    // 4. Accept Additional Content & Append Operation
    if (!appendContent) {
      appendContent = await askQuestion("\\nEnter additional content to append: ");
    }

    console.log(`\\n[STEP 3] Appending content to '${filename}'...`);
    fs.appendFileSync(filePath, "\\n" + appendContent, "utf8");
    console.log(`[SUCCESS] Content appended successfully.`);

    // 5. Read and Display Final Contents
    console.log(`\\n[STEP 4] Reading and displaying final contents of '${filename}'...`);
    const finalContent = fs.readFileSync(filePath, "utf8");
```

Source Code (Continued)

```
    console.log("===== FINAL CONTENTS =====");
    console.log(finalContent);
    console.log("=====");
} catch (err) {
    console.error("\n[ERROR] File System operation failed:", err.message);
} finally {
    rl.close();
}
}

runFileManager();
```

7. CODE EXPLANATION

Method	Description
<code>fs.writeFileSync</code>	Writes data to a file synchronously.
<code>fs.appendFileSync</code>	Appends data to a file synchronously.
<code>fs.readFileSync</code>	Reads file content synchronously.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Create file	Enter filename and content	File created and read successfully	[Execution Verified]	Pass

9. OUTPUT

Execution Output

```
Command Prompt • node fileManager.js student_records.txt

=====
Node.js Interactive File System Manager
=====

[STEP 1] Creating and writing to 'student_records.txt'...
[SUCCESS] File 'student_records.txt' written successfully (37 bytes).

[STEP 2] Reading contents of 'student_records.txt'...
----- Current Content -----
Initial student entry: J Satya Srikar
-----

[STEP 3] Appending content to 'student_records.txt'...
[SUCCESS] Content appended successfully.

[STEP 4] Reading and displaying final contents of 'student_records.txt'...
===== FINAL CONTENTS =====
Initial student entry: J Satya Srikar
Appended record: Course CSM-B 2026
=====
```

Execution Output

10. RESULT

The File System Manager was successfully implemented and tested.

Submission Package Structure:

Observation task 6/Question2/